

OpenClaw Tools Introduction

Tools are the "capability interfaces" that OpenClaw exposes to agents, allowing models to complete real operations in a safer, more structured way, such as reading/writing files, running commands, managing sessions, web access, UI automation, etc. Compared to the old `openc1aw-*` Skills, tools are typed, controllable, and can be precisely limited to "what they are allowed to do / not allowed to do" through configuration.

OpenClaw Tools Introduction

1. Key Concepts
 - 1.1 Relationship Between Tools and Skills
 - 1.2 More Tools Is Not Always Better
 - 1.3 Core of Tool Strategy: profile + allow/deny
2. Quick Start
 - 2.1 Choose a Tool Configuration File (profile)
 - 2.2 Use allow/deny for Precise Control (Recommended to Start from deny)
 - 2.3 Only Tighten for a Specific Provider/Model
3. Tool Groups (group:*)
4. High-Risk Tools: How to Configure Safely
 - 4.1 Command Execution Related (group:runtime)
 - 4.2 apply_patch Tips
 - 4.3 Isolate Tools by Agent (Recommended Practice)
5. Recommended Configuration Templates (Ready to Use)
 - 5.1 Daily Conversation (Safer)
 - 5.2 Learning/Writing Documentation (Allow web search, but no command execution)
 - 5.3 Development Debugging (Can read/write files, but treat command execution as "optional")
6. Common Issues
 - 6.1 Configured allow but it didn't take effect
 - 6.2 A certain model can use tools, but after switching models it doesn't work
 - 6.3 Which profile should I choose?
7. Official Documentation

1. Key Concepts

1.1 Relationship Between Tools and Skills

- Tools: System-level "capability interfaces", typed, configurable, can be limited by provider/model
- Skills: More like "reusable workflows/plugin capabilities". Many old skills will be replaced or weakened by the tool system

If you want your agent to "do things", prioritize tools; if you want to encapsulate a set of processes as reusable capabilities, then consider Skills.

1.2 More Tools Is Not Always Better

The more tools, the more the model can do, but the greater the risk. Best practice: only enable the tools you really need by default, and turn off or open high-risk capabilities (especially command execution) separately as needed.

1.3 Core of Tool Strategy: profile + allow/deny

Tool sets are usually trimmed in this order:

1. `tools.profile` gives a "basic allow set" first
2. If `tools.byProvider` is configured, it further tightens for specified provider or provider/model (it can only narrow the tool set)
3. Finally, use `tools.allow` / `tools.deny` to precisely tighten or loosen (deny takes precedence)

2. Quick Start

OpenClaw's tool allow policy is written in `openclaw.json`. Generally, we don't need manual configuration:

- `deny` has higher priority than `allow`
- Supports `*` wildcards, and matching is case-insensitive

2.1 Choose a Tool Configuration File (profile)

Common profiles (from "least capable" to "most capable"):

- `minimal`: Only `session_status`
- `messaging`: Messaging capabilities + sessions capabilities
- `coding`: Development/engineering capabilities (files, runtime, sessions, memory, images, etc.)
- `full`: Unrestricted (same as not set)

Example: Enable coding configuration file:

```
{
  "tools": {
    "profile": "coding"
  }
}
```

2.2 Use allow/deny for Precise Control (Recommended to Start from deny)

Example: Globally disable `browser` tool:

```
{
  "tools": {
    "deny": ["browser"]
  }
}
```

Example: On the coding basis, disable all runtime tools (exec / bash / process):

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"]
  }
}
```

2.3 Only Tighten for a Specific Provider/Model

You can keep the global default unchanged and only further tighten the tool set for a specific provider or provider/model (it can only narrow, not expand).

Example: Use coding globally, but use minimal for Google Antigravity:

```
{
  "tools": {
    "profile": "coding",
    "byProvider": {
      "google-antigravity": { "profile": "minimal" }
    }
  }
}
```

Example: Further reduce tools for `openai/gpt-5.2`:

```
{
  "tools": {
    "allow": ["group:fs", "group:runtime", "sessions_list"],
    "byProvider": {
      "openai/gpt-5.2": { "allow": ["group:fs", "sessions_list"] }
    }
  }
}
```

3. Tool Groups (group:*)

In `tools.allow` / `tools.deny`, you can use `group:*` to control a group of tools at once:

Group Name	Expands To
<code>group:runtime</code>	exec, bash, process
<code>group:fs</code>	read, write, edit, apply_patch
<code>group:sessions</code>	sessions_list, sessions_history, sessions_send, sessions_spawn, session_status
<code>group:memory</code>	memory_search, memory_get
<code>group:web</code>	web_search, web_fetch
<code>group:ui</code>	browser, canvas
<code>group:automation</code>	cron, gateway
<code>group:messaging</code>	message
<code>group:nodes</code>	nodes
<code>group:openclaw</code>	All built-in OpenClaw tools (excluding provider plugins)

Example: Only allow file tools + browser:

```
{
  "tools": {
    "allow": ["group:fs", "browser"]
  }
}
```

4. High-Risk Tools: How to Configure Safely

4.1 Command Execution Related (`group:runtime`)

`group:runtime` contains execution capabilities (e.g., exec / bash / process). For these capabilities, it is recommended to follow:

- Disabled by default: First `deny: ["group:runtime"]`, open gradually as needed
- Only give to "specific agents": Concentrate high-risk capabilities on dedicated agents
- Only give to "specific provider/model": Open more tools for stable/trusted models

4.2 `apply_patch` Tips

`apply_patch` is a file editing tool; in some versions it is an experimental feature and may require an additional switch to enable (e.g., via `tools.exec.applyPatch.enabled`, and only some models support it). If you find the model clearly can modify files but cannot call `apply_patch`, priority check tool strategy and related switches.

4.3 Isolate Tools by Agent (Recommended Practice)

You can set a global tool strategy and then override for a specific agent, letting "high-permission capabilities" be concentrated on a few agents.

Example: Use coding globally, but the support agent only uses messaging, and additionally allows Slack tools:

```
{
  "tools": { "profile": "coding" },
  "agents": {
    "list": [
      {
        "id": "support",
        "tools": { "profile": "messaging", "allow": ["slack"] }
      }
    ]
  }
}
```

5. Recommended Configuration Templates (Ready to Use)

5.1 Daily Conversation (Safer)

Goal: Chat only + view session status only, don't give it the ability to "mess with the system".

```
{
  "tools": {
    "profile": "minimal"
  }
}
```

5.2 Learning/Writing Documentation (Allow web search, but no command execution)

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"],
    "allow": ["group:web"]
  }
}
```

5.3 Development Debugging (Can read/write files, but treat command execution as "optional")

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"]
  }
}
```

When you really need to execute commands, temporarily open `group:runtime`, then close it again after use.

6. Common Issues

6.1 Configured allow but it didn't take effect

- Check if `deny` simultaneously hits the same tool (deny takes precedence)
- Check if the tool name is spelled correctly (matching is case-insensitive, but the name must exist)
- If `tools.allow` only lists unknown/unloaded plugin tools, the system may ignore the allow list to ensure core tools are available

6.2 A certain model can use tools, but after switching models it doesn't work

- You may have set `tools.byProvider` (provider or provider/model) to make additional restrictions on that model
- Some provider endpoints have inconsistent tool support. It is recommended to use smaller tool sets for unstable endpoints

6.3 Which profile should I choose?

- Only want to chat: `minimal`
- Daily messaging + manage sessions: `messaging`
- File/engineering tasks: `coding`
- You know exactly what you're doing and can bear the risk, I want the complete powerful OpenClaw: `full`

7. Official Documentation

- OpenClaw Tools Documentation: <https://docs.openclaw.ai/zh-CN/tools>